



Instituto Tecnológico de Costa Rica

Escuela de Ingeniería Electrónica

Taller de diseño digital (EL3313)

Prof. Kaleb Alfaro Bonilla, M.Sc.

Estudiantes:

2021155576 - Bolaños Solís Oscar

2022207101 - Bou Espinoza Raquel

2022195304 - Lobo Campos Andrés

2021021770 - Sánchez Herrera Angélica

Lab. 3: Microcontrolador Parte 2

I Semestre 2026

Resumen

En el presente documento se desarrolla el diseño e implementación de un sistema basado en un núcleo RISC-V sobre una FPGA Nexys 4 DDR. El sistema integra una memoria de programa (ROM), una memoria de datos (RAM) y periféricos mapeados en memoria, incluyendo módulos de comunicación UART y SPI, los cuales se interconectan mediante una arquitectura de memoria mapeada.

El procesador ejecuta instrucciones almacenadas en la ROM, permitiendo la interacción con los distintos dispositivos del sistema. Como extensión del laboratorio anterior, se implementó una interfaz SPI para la comunicación con el acelerómetro ADXL362 integrado en la tarjeta Nexys 4 DDR. A través de este periférico es posible adquirir las lecturas de aceleración correspondientes a los ejes X, Y y Z, las cuales son posteriormente transmitidas hacia una computadora mediante la interfaz UART.

Durante el desarrollo se abordaron aspectos clave como la decodificación de direcciones, la integración de nuevos periféricos, la implementación del protocolo SPI, el acceso a registros internos del acelerómetro y la sincronización entre los distintos bloques del sistema. Asimismo, se desarrolló una aplicación de software ejecutada sobre el procesador RISC-V encargada de controlar la adquisición de datos y la comunicación con la computadora anfitriona.

La correcta operación del sistema fue verificada mediante simulaciones funcionales en Vivado, pruebas unitarias de los módulos desarrollados mediante testbenches y validación experimental sobre la FPGA Nexys 4 DDR. Adicionalmente, se utilizó la herramienta Release Beta 5.5 para comprobar la transmisión serial y la recepción de las lecturas del acelerómetro en tiempo real.

Tabla de contenidos

Resumen.....	2
Marco Teórico	4
Introducción.....	5
Diseño del sistema	5
Descripción de módulos	7
Mapa de memoria	12
Funcionamiento del sistema	13
Aplicaciones externas.....	14
Implementación en FPGA.....	18
Verificación de funcionamiento en la FPGA	19
Conclusiones	19
Referencias	21

Marco Teórico

Protocolo SPI

El protocolo SPI (Serial Peripheral Interface) es un estándar de comunicación serial síncrona utilizado para la conexión de microcontroladores y sistemas digitales con periféricos como sensores, memorias y convertidores de datos. SPI emplea una arquitectura maestro-esclavo y utiliza cuatro señales principales: SCLK (reloj), MOSI (datos del maestro al esclavo), MISO (datos del esclavo al maestro) y CS (selección del dispositivo) [1]. La transmisión se realiza de forma síncrona, donde cada pulso de reloj permite el intercambio simultáneo de un bit entre maestro y esclavo. Su alta velocidad, simplicidad y bajo costo de implementación lo convierten en una de las interfaces más utilizadas en sistemas embebidos. La operación del protocolo se controla mediante registros de configuración que permiten habilitar el módulo, ajustar la frecuencia del reloj y definir el modo de comunicación [1].

Sensor ADXL362

El ADXL362 es un acelerómetro MEMS digital de tres ejes y bajo consumo energético que permite medir aceleraciones en las direcciones X, Y y Z mediante una interfaz SPI [2]. El dispositivo incorpora registros internos para configuración y adquisición de datos, entre los que destacan POWER_CTL para habilitar el modo de medición y FILTER_CTL para configurar el rango de operación y la frecuencia de muestreo [2]. Las mediciones de aceleración se almacenan en registros dedicados para cada eje y se representan mediante valores de 16 bits en complemento a dos. Estas lecturas permiten determinar cambios de movimiento, vibraciones y orientación espacial del dispositivo [2].

El ADXL362 integra tres sensores orientados ortogonalmente, permitiendo medir la aceleración en los ejes X, Y y Z [2]. Cada eje genera una señal proporcional a la aceleración experimentada en su dirección correspondiente. Cuando el sensor se encuentra en reposo, la aceleración gravitacional se distribuye entre los tres ejes según la orientación física del dispositivo. Por esta razón, las variaciones en las lecturas permiten identificar inclinaciones, movimientos y cambios de posición. El análisis conjunto de los tres ejes proporciona una representación tridimensional del movimiento del sistema [2].

Protocolo AXI-Lite

AXI-Lite (Advanced eXtensible Interface Lite) es una versión simplificada del protocolo AXI utilizada para la comunicación entre procesadores y periféricos en sistemas digitales [3]. A diferencia de AXI-Full, está orientado a transferencias simples de lectura y escritura, sin soporte para transferencias en ráfaga, reduciendo así la complejidad del hardware.

Introducción

En este laboratorio se desarrolló e implementó una extensión del sistema basado en un núcleo RISC-V de 32 bits sobre una FPGA Nexys 4 DDR desarrollado en el laboratorio anterior. Además de los módulos previamente implementados, se incorporó una interfaz de comunicación SPI para interactuar con el acelerómetro ADXL362 integrado en la tarjeta, permitiendo la adquisición de datos de movimiento en tiempo real.

El objetivo principal consiste en integrar los distintos módulos de hardware, incluyendo memoria, periféricos y lógica de control, para lograr la lectura de los datos del acelerómetro mediante el protocolo SPI y su posterior transmisión hacia una computadora a través de una interfaz UART. De esta manera, la FPGA puede funcionar como un dispositivo de control externo para aplicaciones ejecutadas en la computadora anfitriona.

Asimismo, se implementó el software necesario para ejecutar sobre el procesador RISC-V, encargado de controlar la comunicación con el acelerómetro, procesar las lecturas de los ejes X, Y y Z, y transmitir dicha información mediante la interfaz serial. Finalmente, el funcionamiento del sistema fue verificado mediante simulaciones, testbenches para los módulos desarrollados y pruebas experimentales utilizando herramientas como Vivado y Tera Term.

Diseño del sistema

El diseño completo puede entenderse como una arquitectura de adquisición y transmisión de datos basada en un microprocesador embebido. El flujo de operación inicia con el reloj de 100 MHz de la FPGA, el cual es estabilizado mediante el bloque `clk_wiz_0`. Con este reloj trabajan el procesador PicoRV32, las memorias y los periféricos.

El procesador ejecuta el programa almacenado en ROM. Desde el software, se activa una lectura del acelerómetro escribiendo en el registro de control del periférico SPI. El módulo SPI se comunica físicamente con el ADXL362 mediante las señales SCLK, MOSI, MISO y CS. Luego, los datos digitales de aceleración son almacenados en registros internos y puestos a disposición del procesador en direcciones de memoria.

Después de leer los datos, el procesador los separa en los valores X, Y y Z. Estos valores son enviados por UART hacia una computadora o terminal serial, permitiendo visualizar las mediciones del acelerómetro. Al mismo tiempo, los LEDs de la FPGA se utilizan como indicadores de estado del sistema. Por ejemplo, pueden indicar inicialización, operación correcta o error en la lectura SPI.

En términos funcionales, el sistema está formado por cinco bloques principales: procesamiento, memoria, comunicación con periféricos, adquisición SPI y visualización/salida. El bloque de

procesamiento está compuesto por el PicoRV32; el bloque de memoria por ROM y RAM; el bloque de comunicación por el puente AXI, UART y GPIO; el bloque de adquisición por el controlador SPI del ADXL362; y el bloque de salida por los LEDs y la transmisión UART.

Esta organización permite que el sistema sea modular, ya que cada periférico se encuentra separado del procesador y se comunica mediante direcciones definidas. Además, facilita la depuración, porque el estado del sistema puede observarse tanto en los LEDs como en la salida serial UART.

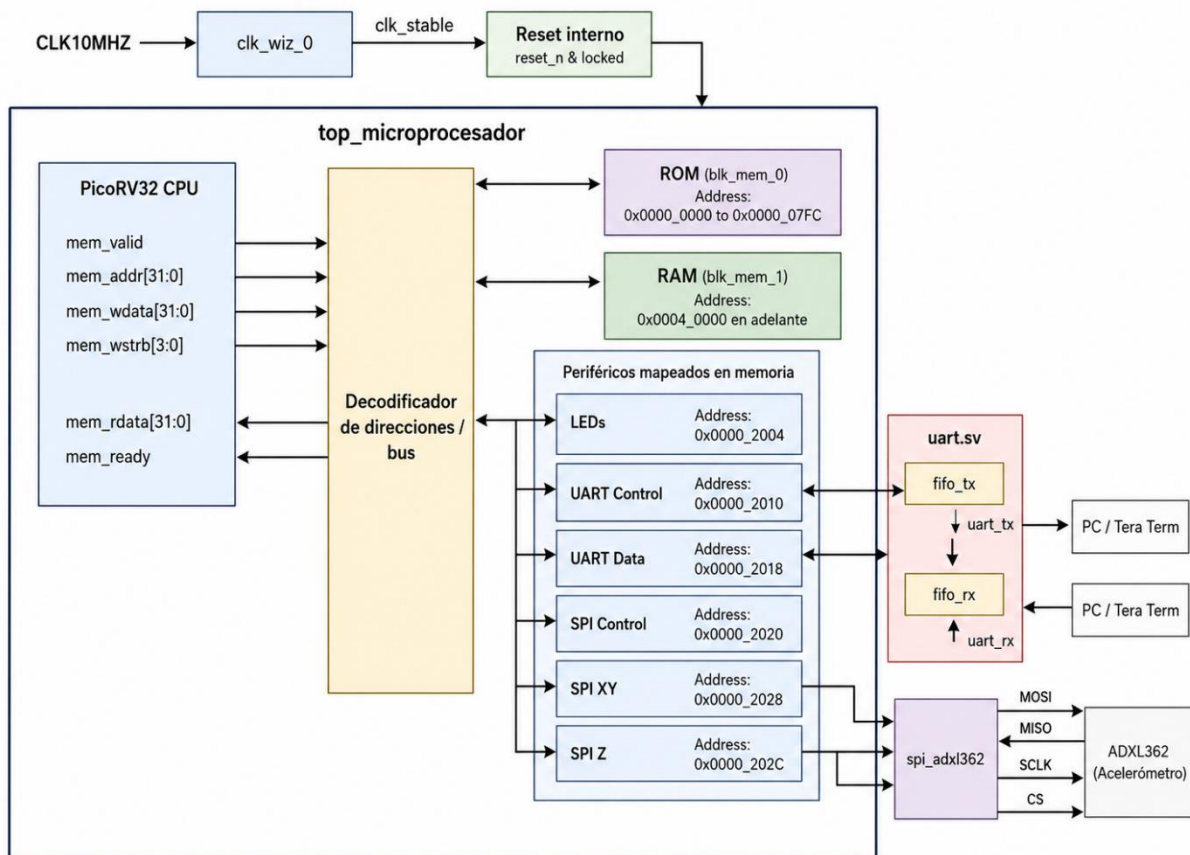


Figura1. Diagrama de bloques del microcontrolador basado en PicoRV32 con periféricos mapeados en memoria.

Descripción de módulos

Descripción general del sistema

El sistema desarrollado corresponde a una arquitectura digital basada en un microprocesador RISC-V implementado en FPGA. El diseño utiliza el núcleo PicoRV32 como unidad central de procesamiento, encargado de ejecutar un programa en C almacenado en memoria ROM. A partir de este programa, el procesador accede a distintos periféricos mediante direcciones mapeadas en memoria, permitiendo controlar salidas digitales, comunicarse por UART y obtener datos de un acelerómetro mediante SPI. Repositorio: https://github.com/Oscar-Electronics/Taller_diseño_digital/tree/main/Laboratorio%203

El módulo principal del sistema es `top_microprocesador.v`, el cual integra todos los bloques funcionales del proyecto. En este módulo se instancia el reloj del sistema, el procesador PicoRV32, la memoria ROM, la memoria RAM, el puente de comunicación hacia periféricos AXI, el GPIO para LEDs, el periférico UART y el módulo SPI para el acelerómetro ADXL362. Además, este módulo realiza la decodificación de direcciones para seleccionar qué bloque debe responder a cada acceso de memoria realizado por el procesador.

La arquitectura utiliza un esquema de memoria mapeada. Esto significa que el procesador no se comunica con los periféricos mediante instrucciones especiales, sino escribiendo o leyendo direcciones específicas. Por ejemplo, el programa en C define registros para LEDs, UART y SPI en las direcciones 0x2004, 0x2010, 0x2018, 0x2020, 0x2028 y 0x202C. De esta forma, el software puede activar lecturas SPI, obtener los datos de los ejes X, Y y Z, enviar información por UART y mostrar estados en los LEDs.

Módulo Top microprocesador

El módulo `top_microprocesador.v` funciona como el nivel superior del diseño. Su tarea principal es conectar el reloj físico de la FPGA, el reset, los pines UART, los LEDs y las señales SPI del acelerómetro con los módulos internos del sistema. Dentro de este bloque se instancia el `clk_wiz_0`, que genera un reloj estable para el procesador y los periféricos. También se instancia el núcleo `picorv32`, configurado con una dirección inicial de pila en 0x0004_07FC.

Este módulo también contiene la lógica de decodificación de direcciones. Las direcciones bajas se usan para acceder a la ROM, el rango 0x0004_0000 a 0x0004_1FFC se utiliza como RAM, las direcciones 0x2004, 0x2010, 0x2018 y 0x201C corresponden a periféricos AXI, y las direcciones

0x2020, 0x2028 y 0x202C corresponden al periférico SPI del acelerómetro. Según la dirección solicitada por el procesador, el módulo selecciona la respuesta de ROM, RAM, AXI o SPI.

En resumen, este módulo representa la integración completa del sistema, ya que une el procesador, las memorias y los periféricos necesarios para interactuar con el acelerómetro y mostrar los resultados.

Núcleo picorv32.v

El archivo picorv32.v contiene el procesador RISC-V utilizado como CPU del sistema. Este procesador ejecuta el programa compilado a partir del archivo main.c. El núcleo genera señales de acceso a memoria como mem_valid, mem_addr, mem_wdata, mem_wstrb y recibe como respuesta mem_ready y mem_rdata.

En el diseño, el procesador no distingue directamente entre memoria y periféricos. Simplemente realiza lecturas y escrituras a direcciones de 32 bits. La lógica externa del módulo superior es la encargada de interpretar esas direcciones y conectar al procesador con la ROM, RAM, GPIO, UART o SPI.

Memoria ROM y RAM

El sistema incluye una ROM interna implementada como un arreglo de registros de 32 bits. Esta memoria se inicializa mediante \$readmemh, cargando un archivo hexadecimal que contiene el programa compilado. La ROM ocupa el rango de direcciones iniciales y sirve para almacenar las instrucciones ejecutadas por el procesador.

También se implementa una RAM interna en el rango de direcciones 0x0004_0000 a 0x0004_1FFC. Esta memoria permite almacenar datos temporales durante la ejecución del programa. La RAM soporta escritura por bytes mediante la señal mem_wstrb, lo cual permite modificar únicamente los bytes seleccionados dentro de una palabra de 32 bits.

Módulos uart, uart_tx, uart_rx y FIFO

El módulo uart es el bloque general de comunicación serial. Este se encarga de conectar el transmisor uart_tx, el receptor uart_rx y dos FIFOs internas: una para transmisión y otra para recepción. Las FIFOs permiten almacenar temporalmente los datos enviados o recibidos, evitando que se pierdan caracteres cuando el procesador y la línea serial trabajan a ritmos diferentes.

El módulo uart_tx implementa la transmisión serial. Toma un byte desde la FIFO de transmisión y lo envía bit por bit siguiendo el formato UART, incluyendo bit de inicio, bits de datos y bit de

parada. Por otro lado, el módulo `uart_rx` implementa la recepción serial. Este bloque detecta el bit de inicio, muestrea los bits recibidos y valida el bit de parada. Además, utiliza una estrategia de muestreo por mayoría para mejorar la robustez frente a ruido.

Los archivos `fifo.sv`, `fifo_ctrl.sv` y `fifo_mem.sv` implementan la estructura de almacenamiento temporal. `fifo.sv` integra el controlador y la memoria; `fifo_ctrl.sv` maneja los punteros de lectura y escritura, además de las señales de lleno y vacío; y `fifo_mem.sv` contiene la memoria interna de la FIFO.

Módulo `axi_bridge`

El módulo `axi_bridge` funciona como un puente entre la interfaz de memoria simple del PicoRV32 y los periféricos que usan una interfaz tipo AXI-Lite. Este bloque recibe las señales del procesador, identifica si la operación es de lectura o escritura y genera las señales AXI correspondientes para el periférico seleccionado.

El puente puede dirigir accesos hacia el GPIO o hacia la UART. Para esto, utiliza la dirección solicitada por la CPU. La dirección `0x0000_2004` se asocia con el GPIO, mientras que las direcciones `0x0000_2010`, `0x0000_2018` y `0x0000_201C` se asocian con la UART. Internamente, el módulo utiliza una máquina de estados con etapas para escritura, espera de respuesta, lectura y recepción de datos.

Este bloque es importante porque permite que el procesador trabaje con una interfaz simple de memoria, mientras que los periféricos mantienen una estructura de comunicación tipo AXI.

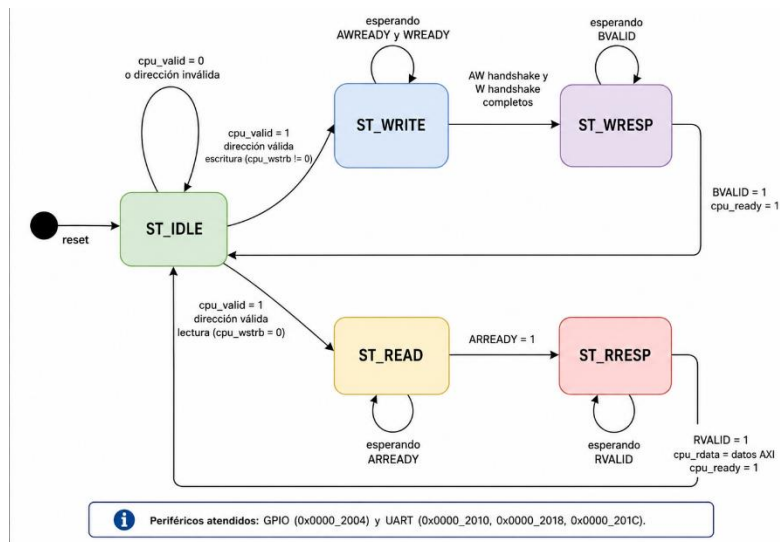


Figura 2. Diagrama de máquina de estados del módulo `axi_bridge`

Módulo axi_gpio_slave

El módulo axi_gpio_slave implementa un periférico GPIO controlado mediante AXI-Lite. Su función principal es almacenar un dato de 32 bits en un registro interno llamado gpio_reg. Los 16 bits menos significativos de este registro se conectan directamente a la salida leds.

Cuando el procesador escribe en la dirección del GPIO, el dato enviado por la CPU se guarda en el registro interno. De esta forma, el software puede controlar el estado de los LEDs de la FPGA. En el programa principal, los LEDs se utilizan como indicadores visuales del estado del sistema, por ejemplo, para mostrar inicio, operación correcta o error.

Módulo axi_uart_slave

El módulo axi_uart_slave permite que el procesador envíe y reciba datos mediante comunicación serial UART. Este módulo expone registros mapeados en memoria para control y datos. El registro de control se encuentra en 0x2010, mientras que los registros de datos se encuentran en 0x2018 y 0x201C.

Internamente, este módulo instancia el bloque uart, configurado con una frecuencia de sistema de 100 MHz y una velocidad de transmisión de 9600 baudios. El software escribe un carácter en el registro de datos y luego activa el registro de control para iniciar la transmisión. Esto permite enviar por el puerto serial los valores obtenidos del acelerómetro.

Módulo adxl362_controller

El módulo adxl362_controller se encarga de controlar la comunicación SPI con el acelerómetro ADXL362 para inicializarlo y leer sus datos de aceleración en los ejes X, Y y Z. Al salir del reinicio, el módulo espera un tiempo inicial y luego configura el acelerómetro escribiendo el valor 0x02 en el registro POWER_CTL (0x2D), lo que activa el modo de medición. Después de esta configuración, el sistema queda en estado de espera hasta recibir la señal start_read. Cuando esta señal se activa, el módulo baja la señal spi_cs_n, envía el comando de lectura 0x0B junto con la dirección inicial 0x0E, y posteriormente recibe los bytes correspondientes a los datos de aceleración. Una vez finalizada la lectura, combina los bytes bajos y altos de cada eje, aplica extensión de signo a 12 bits para obtener valores con signo de 16 bits, actualiza las salidas x_data, y_data y z_data, activa la señal data_ready e indica que el módulo ya no está ocupado mediante busy = 0.

Módulo spi_adxl362

El módulo spi_adxl362 funciona como una interfaz entre el CPU/sistema principal y el acelerómetro ADXL362 conectado por SPI. Este módulo expone registros mapeados en memoria para que el procesador pueda iniciar una lectura, consultar el estado del acelerómetro y obtener los datos de los ejes X, Y y Z. Internamente, separa la lógica en dos bloques principales: un controlador adxl362_controller, encargado de inicializar el acelerómetro y solicitar lecturas de datos, y un spi_master, encargado de generar las señales físicas del protocolo SPI, como spi_sclk, spi_mosi, spi_miso y spi_cs_n. Cuando el CPU escribe en el registro de control, se activa start_read, el controlador realiza la lectura del acelerómetro y, al finalizar, los datos quedan disponibles en los registros de salida para ser leídos por el sistema. Además, el módulo entrega señales de estado como busy y data_ready para indicar si la lectura está en proceso o si los datos ya están listos.

Módulo spi_master

El módulo spi_master implementa un maestro SPI genérico encargado de transmitir y recibir un byte por comunicación serial síncrona. Cuando recibe la señal start y el módulo no está ocupado, carga el byte de transmisión tx_byte, activa la señal busy y comienza a generar el reloj SPI sclk a partir del reloj principal clk, usando el parámetro DIVIDER para ajustar la frecuencia según CLK_FREQ y SPI_FREQ. Durante la transferencia, el módulo envía los bits por mosi desde el bit más significativo hasta el menos significativo, mientras captura simultáneamente los bits recibidos por miso en un registro de desplazamiento. Al completar los 8 bits, guarda el byte recibido en rx_byte, desactiva busy, activa brevemente la señal done para indicar que la transferencia terminó y queda listo para iniciar una nueva operación SPI.

Programa main.c

El archivo main.c contiene el programa ejecutado por el procesador PicoRV32. En este programa se definen macros para acceder a los periféricos mediante punteros a direcciones específicas. Por ejemplo, LEDS apunta a 0x2004, UART_CTRL a 0x2010, UART_DATA0 a 0x2018, SPI_CTRL a 0x2020, SPI_XY a 0x2028 y SPI_Z a 0x202C.

El programa inicializa los LEDs, envía el mensaje “ADXL362 XYZ” por UART y luego entra en un ciclo infinito. En cada iteración, solicita una lectura SPI escribiendo en SPI_CTRL, espera a que el módulo SPI indique que terminó la lectura y luego obtiene los valores de X, Y y Z. Si la lectura es correcta, muestra un patrón en los LEDs y transmite los valores por UART. Si ocurre un error de comunicación o un timeout, enciende un patrón de error y envía el mensaje “SPI ERROR”.

Archivo build2.ps1

El archivo `build2.ps1` corresponde al script de compilación del firmware del sistema. Su función es automatizar el proceso de construcción del programa que será ejecutado por el procesador PicoRV32. Este script compila el archivo de arranque `start.s` y el programa principal `main.c` utilizando la herramienta `riscv-none-elf-gcc`, ambos configurados para la arquitectura RV32I.

Después de compilar los archivos fuente, el script enlaza los objetos generados mediante `riscv-none-elf-ld`, ubicando la sección de código `.text` a partir de la dirección `0x00000000`, que corresponde al inicio de la memoria ROM del sistema. Posteriormente, el archivo ejecutable `.elf` se convierte a formato binario y luego a un archivo hexadecimal `.hex`, el cual es leído por la memoria ROM del diseño en Vivado mediante `$readmemh`.

En resumen, `build2.ps1` permite generar automáticamente el archivo de firmware que será cargado en la ROM del microcontrolador, evitando realizar manualmente cada paso de compilación, enlace y conversión.

Archivo `start.s`

El archivo `start.s` corresponde al código de arranque del sistema. Su función es inicializar el puntero de pila y luego llamar a la función `main`. Una vez que el programa principal finaliza, el código entra en un ciclo infinito de parada. Este archivo es necesario porque el procesador ejecuta instrucciones desde cero y requiere una secuencia mínima de inicialización antes de ejecutar código en C.

Mapa de memoria

El sistema utiliza un esquema de memoria mapeada, donde cada periférico ocupa un rango específico de direcciones dentro del espacio de memoria del procesador.

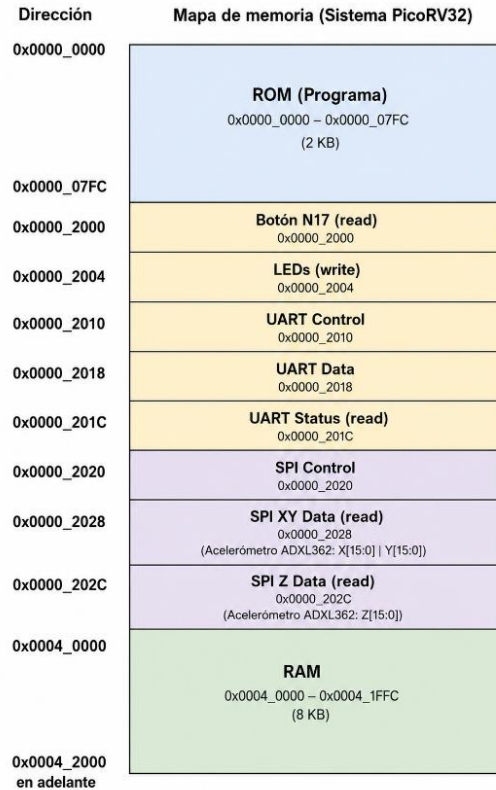


Figura 3. Mapa de memoria

Funcionamiento del sistema

Al iniciar el sistema, el procesador PicoRV32 comienza la ejecución del programa almacenado en la memoria ROM, accediendo a las instrucciones mediante la interfaz de memoria. Durante la secuencia de arranque se inicializan los periféricos del sistema y se configura el acelerómetro ADXL362 a través del controlador SPI. Una vez completada esta etapa, el procesador indica mediante los LEDs que el sistema se encuentra listo para operar.

La comunicación entre el procesador y los distintos módulos se realiza mediante la interfaz de memoria, utilizando señales como `mem_valid`, `mem_addr`, `mem_wdata`, `mem_wstrb`, `mem_rdata` y `mem_ready`. Cuando el procesador desea acceder a un dispositivo, activa la señal `mem_valid` junto con la dirección correspondiente y espera la confirmación de la operación mediante `mem_ready`.

El módulo superior se encarga de decodificar la dirección generada por el procesador y seleccionar el dispositivo adecuado, ya sea la ROM, la RAM o alguno de los periféricos mapeados en memoria. En las operaciones de lectura, el dispositivo seleccionado responde mediante `mem_rdata`, mientras que en las operaciones de escritura los datos son enviados por medio de `mem_wdata`.

La interacción con la computadora se realiza mediante la interfaz UART. El programa ejecutado por el procesador permanece monitoreando la recepción de datos a través de este periférico. Cuando se recibe un comando de inicio desde la computadora, el sistema comienza el proceso de adquisición de datos del acelerómetro.

Para obtener las mediciones, el procesador solicita una nueva lectura al controlador SPI mediante el registro de control correspondiente. El periférico SPI establece la comunicación con el ADXL362, realiza la lectura de los registros asociados a los ejes X, Y y Z, y almacena los resultados en registros internos accesibles por el procesador. Posteriormente, el software lee estos valores y los prepara para su transmisión.

Las muestras obtenidas del acelerómetro son enviadas periódicamente hacia la computadora mediante la interfaz UART. Este proceso continúa de forma indefinida mientras el sistema permanezca en modo de adquisición. Cuando se recibe un comando de finalización desde la computadora, el procesador detiene la transmisión de datos y retorna al estado de espera, listo para iniciar un nuevo ciclo de adquisición cuando sea solicitado.

De esta manera, el sistema permite la ejecución continua de un programa almacenado en ROM, la adquisición de información proveniente de sensores externos mediante SPI y la comunicación bidireccional con una computadora mediante UART, integrando todos los componentes a través de una arquitectura de memoria mapeada.

Aplicaciones externas

Aplicación para la Interfaz: accel to controlV0

La aplicación “accel to controlV0” es un programa que recibe los datos del acelerómetro a través de la interfaz UART para dar como salida el uso de una tecla del teclado o movimientos del mouse dependiendo del tipo de control seleccionado.

Para lograr esto se necesita que los datos se reciban en el formato “X#### Y#### Z####”, porque el programa utiliza el símbolo “(espacio)” para separar las variables en X, Y y Z; después utiliza el símbolo “=” para empezar a recolectar los datos de cada aceleración.

Se utilizan las siguientes fórmulas para calcular las velocidades, los ángulos y la magnitud del movimiento de la FPGA:

aceleración

$$velocidad_x(k) = velocidad_x(k - 1) + aceleración_x(k)\Delta t$$

$$velocidad_y(k) = velocidad_y(k - 1) + aceleración_y(k)\Delta t$$

$$velocidad_z(k) = velocidad_z(k - 1) + aceleración_z(k)\Delta t$$

Donde k es la muestra actual.

$$pitch = \tan^{-1} \left(\frac{aceleración_y}{\sqrt{aceleración_x^2 + aceleración_z^2}} \right) * \frac{180}{\pi}$$

$$roll = \tan^{-1} \left(\frac{-aceleración_x}{\sqrt{aceleración_y^2 + aceleración_z^2}} \right) * \frac{180}{\pi}$$

$$magnitud = \sqrt{aceleración_x^2 + aceleración_y^2 + aceleración_z^2}$$

Para los controles se utilizan los ángulos y la magnitud de las aceleraciones; aunque no se utiliza la velocidad esta se calcula, pero no se usa ya que esta es un dato no confiable por varias razones, como el hecho que los acelerómetros tienden a tener drift, también hay que tener en cuenta que la forma en la que se está calculando lleva a que estos números sigan creciendo sin fin, ya que no se reinicia la cuenta cuando la FPGA se encuentra en reposo.

El esquema de control se determina de la siguiente forma

Para control WASD y por flechas:

Comandos o teclas	Ángulos ° de activación		Ángulos ° de desactivación		Magnitud de activación	Magnitud de desactivación
	Pitch	Roll	Pitch	Roll		
W o ↑	<-30		>-15			
S o ↓	>30		<15			
A o ←		>40		<20		
D o →		<-40		>-20		

Espacio					<500	>450
Centro	>10	>10	<10	<10		

Para control de mouse se utiliza otro método, ya que al mouse solo se le pueden dar coordenadas X y Y, primero se obtiene la posición del mouse en la pantalla, luego si la FPGA se encuentra fuera de la zona del centro, se convierte el ángulo de la FPGA en un movimiento, finalmente se decide una sensibilidad para el mouse, en este caso es 0.5, y se calculan las tasas de cambio como:

$$dx = -Roll * sensibilidad$$

$$dy = Pitch * sensibilidad$$

Seguidamente, con estos movimientos se guardan como una variable int para conseguir un desplazamiento en pixeles, finalmente se le envía al mouse la instrucción de ir a la posición $(posición_x, posición_y) + (d_x, d_y)$.

Para utilizar la aplicación se deben de seguir los siguientes pasos

- 1 - Descardar la carpeta comprimida de Release beta 6.
- 2- Descomprimir la carpeta en la computadora del usuario.
- 3- Hacer doble click en “accel a control v0.exe), cuando aparezca la pantalla de Windows porque el archivo no tiene la firma digital de un programa normal se debe dar click en aceptar.
- 4- Una vez que inicia la app, se presiona el botón “Buscar COM” para buscar los puertos COM abiertos.
- 5- Seleccionar el puerto COM correspondiente en el menú de drop down a la par del botón.
- 6- Seleccionar el baud rate que corresponde a su programa, en este caso son 9600 baudios.
- 7- Hacer click en el botón “Conectar” para iniciar la conexión.
- 8- Cuando se desee empezar el acelerómetro se da click en el botón “Iniciar Acelerómetro” y cuando se desee detenerlo se hace click en el botón “Detener Acelerómetro”
- 9- Cuando se desee iniciar el control se selecciona le modo de control deseado: Control WASD Legacy para que las teclas presionadas sean W, A, S, D y espacio; Control Flechas para las teclas de las flechas del teclado y espacio; y Control Mouse es para controlar el mouse dependiendo del ángulo de la FPGA.
- 10- Cuando se desee cerrar la conexión por el puerto COM de debe de presionar el botón “Desconectar”

11- El botón “Limpiar monitor” borra todas las líneas de datos guardadas en el monitor serial, el monitor serial también se limpia cuando llega a las 2000 líneas.

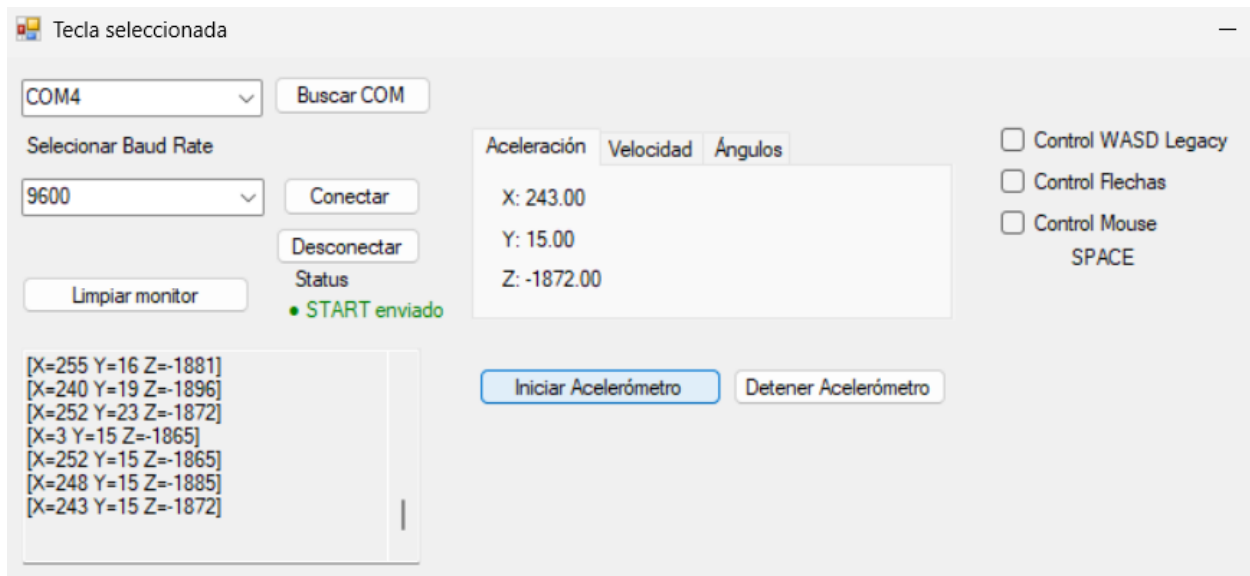


Figura 4. Interfaz accel to controlV0

Juego Conejo Matrero

El juego Conejo Matrero es una adaptación del juego space invaders, en la que el jugador controla un conejo encargado de defenderse de oleadas de abejas que avanzan progresivamente por la pantalla. El personaje puede desplazarse horizontalmente y lanzar zanahorias para eliminar a los enemigos, acumulando puntos por cada abeja derrotada. Conforme el jugador avanza y elimina todas las abejas de una ronda, se genera una nueva oleada con una mayor velocidad de movimiento, incrementando gradualmente la dificultad y manteniendo el desafío durante la partida.

El proyecto fue desarrollado utilizando el lenguaje de programación Python y la biblioteca Pygame, incorporando gráficos tipo pixel art para representar los personajes, proyectiles, explosiones y escenarios. Además de su funcionamiento mediante teclado, el juego fue concebido como una plataforma para la integración con una FPGA Nexys 4DDR y un acelerómetro

ADXL362, permitiendo que las coordenadas obtenidas por el sensor sean utilizadas para controlar el movimiento y las acciones del personaje. De esta manera, el videojuego no solo cumple una función recreativa, sino que también sirve como demostración práctica de la comunicación entre hardware y software en sistemas digitales embebidos.



Figura 5. Juego Conejo Matrero

Implementación en FPGA

El diseño fue implementado en una FPGA Nexys 4DDR utilizando la herramienta Vivado. Se integraron módulos en Verilog y SystemVerilog para el resto del sistema.

Las restricciones de pines fueron definidas mediante el archivo de constraints .xdc, incluyendo la configuración del reloj, la interfaz UART y señales de control.

Las siguientes imágenes muestran el correcto funcionamiento de la simulación del programa en Vivado. Se prueba mediante dos testbench donde se puede ver la lógica de las señales. El primer testbench fue del módulo spi_adxl362 como se ve en la Figura 6. El segundo testbench fue de xyz, con start y finish como se ve en la Figura 7.

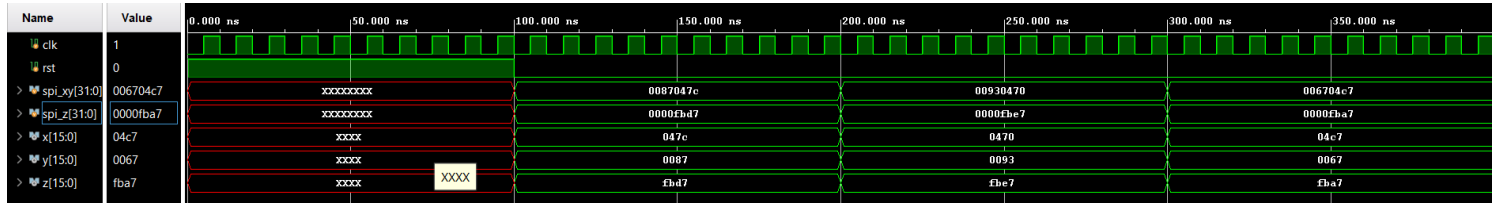


Figura 6. Testbench del módulo spi_adxl362.

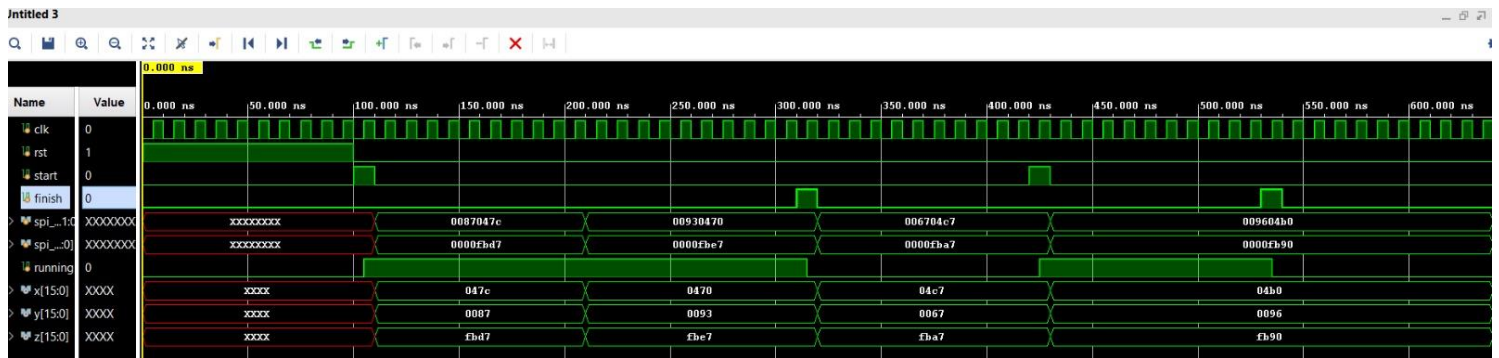


Figura 7. Testbench xyz, con start y finish

Verificación de funcionamiento en la FPGA

Una vez obtenido el resultado esperado en el osciloscopio de Vivado, se procede a generar el bitstream y la implementación en la FPGA. En los siguientes enlaces se puede verificar la funcionalidad de la implementación realizada:

<https://youtu.be/RuhyecXBS04> <https://youtube.com/shorts/sNchcyQjmj8>

Conclusiones

Se logró implementar exitosamente una extensión del sistema basado en un núcleo RISC-V desarrollado en el laboratorio anterior, incorporando una interfaz de comunicación SPI para interactuar con el acelerómetro ADXL362 integrado en la FPGA Nexys 4 DDR. El sistema fue capaz de ejecutar el programa almacenado en memoria, adquirir datos de aceleración

correspondientes a los ejes X, Y y Z, y transmitir dicha información hacia una computadora mediante la interfaz UART.

Uno de los principales retos del proyecto fue la integración del controlador SPI dentro de la arquitectura de memoria mapeada existente, así como la correcta interpretación de los registros del acelerómetro y la sincronización de la comunicación entre el procesador, los periféricos y el sensor. Asimismo, fue necesario verificar cuidadosamente el funcionamiento de cada módulo mediante simulaciones y testbenches antes de realizar la integración completa del sistema.

La implementación permitió profundizar en conceptos relacionados con arquitecturas embebidas, comunicación entre procesadores y periféricos, protocolos seriales de comunicación y diseño modular en FPGA.

Finalmente, se obtuvo una plataforma funcional capaz de adquirir información del sensor físico y transmitirla a aplicaciones externas, demostrando la capacidad de integrar hardware y software dentro de un mismo sistema computacional basado en FPGA.

Referencias

[1] Digi, «¿Qué es una interfaz SPI (Serial Peripheral Interface)? | Definición». Disponible en: <https://es.digi.com/resources/definitions/spi>

[2] Analog Devices, Inc., “ADXL362: Micropower, 3-axis, ± 2 g/ ± 4 g/ ± 8 g digital output MEMS accelerometer,” ADXL362 Data Sheet, Rev. B, 2019. [Online]. Available: <https://www.analog.com/media/en/technical-documentation/data-sheets/ADXL362.pdf>

[3] S. Singh, «AXI-Full and AXI-Lite Interfaces», Logic Fruit Technologies, 23-May-2017. Disponible en: <https://www.logic-fruit.com/blog/digital-interfaces/axi-full-axi-lite-interfaces/>

[4] Digilent, «digilent-xdc/Nexys-4-DDR-Master.xdc at master · Digilent/digilent-xdc», GitHub. Disponible en: <https://github.com/Digilent/digilent-xdc/blob/master/Nexys-4-DDR-Master.xdc>

[5] Oskarwires, «GitHub - oskarwires/uart2: SystemVerilog UART transceiver», GitHub. Disponible en: <https://github.com/oskarwires/uart2.git>